

Weather Risk Dashboard

Escopo do Sistema para Teste Técnico

Backend Python React ou Next.js PostgreSQL + ORM Agente com LLM Deploy em servidor

Prazo de entrega

Quarta-feira, 29/04/2026, até 23h59. A aplicação deve estar publicada online no servidor informado e o código deve estar disponível em repositório Git.

Resumo da proposta

Construir uma plataforma web para acompanhar dados climáticos de cidades monitoradas e da localização atual do usuário, com dashboard, controle de acesso e agente em chat integrado a interface.

Expectativa

Não esperamos uma solução enorme. Esperamos uma solução objetiva, funcional, bem explicada e com decisões técnicas coerentes.

0. Índice

- Contexto e objetivo do sistema
- API externa e localização atual do usuário
- Cidades monitoradas, login e controle de acesso
- Tecnologias, arquitetura, usabilidade e UX
- Dashboard, métricas climáticas e risco operacional
- Páginas esperadas e agente climático
- Publicação, repositório Git, prazo e entregáveis
- Critérios de aceite, diferenciais e observações finais

1. Contexto

Somos uma empresa que opera em diferentes cidades e precisamos acompanhar condições climáticas que podem impactar nossas operações diárias. Hoje, esse acompanhamento é feito de forma manual, consultando previsões em diferentes fontes, o que dificulta a comparação entre localidades, a tomada de decisão e a identificação rápida de riscos.

Queremos uma plataforma web interna chamada Weather Risk Dashboard, capaz de consolidar informações climáticas relevantes, apresentar indicadores claros e permitir que diferentes tipos de usuários acompanhem o cenário climático de forma simples, visual e confiável.

Este escopo será usado como base para um teste técnico. Esperamos que o candidato proponha a melhor solução técnica para atender aos requisitos descritos, tomando decisões coerentes de arquitetura, organização de código, experiência do usuário, integração com API externa, uso de agente e publicação da aplicação em servidor.

Intencao do teste

Embora o escopo possa parecer amplo, a intencao é avaliar a capacidade do candidato de interpretar uma necessidade de negócio, priorizar funcionalidades, tomar decisões técnicas e entregar uma solução funcional.

2. Objetivo do sistema

O sistema deve permitir que usuários autenticados acompanhem dados climáticos de cidades monitoradas, incluindo a localização atual do usuário, e visualizem métricas, rankings, previsões e alertas operacionais.

Além do dashboard, o sistema deve contar com um agente interno em formato de chat, disponível dentro da interface, para responder perguntas sobre os dados climáticos apresentados.

O sistema deverá estar publicado online no servidor informado para que possa ser testado e avaliado.

3. API externa

O sistema deve consumir dados climáticos da API pública Open-Meteo. A documentação oficial da API deve ser usada como referência pelo candidato.

Fonte permitida para consulta

Documentação oficial da Open-Meteo: <https://open-meteo.com/en/docs>

O frontend não deve consumir a API externa diretamente. A aplicação deve ter um backend próprio, responsável por buscar, tratar, organizar e disponibilizar os dados para o restante do sistema.

O candidato deve estudar a documentação da API, entender como obter os dados necessários e propor a melhor forma de integração.

4. Localização atual do usuário

Após o login, o sistema deve solicitar permissão no navegador para acessar a localização atual do usuário.

Quando o usuário permitir o acesso a localização, o sistema deve usar essa posição para carregar informações climáticas da região atual e exibir esses dados no dashboard.

Caso o usuário negue a permissão ou o navegador não consiga obter a localização, o sistema deve continuar funcionando normalmente, exibindo as cidades padrão monitoradas e informando de forma clara que a localização atual não está disponível.

Com a localização atual, o sistema deve apresentar

- Condição climática da região do usuário.
- Temperatura atual ou prevista.
- Probabilidade de chuva.
- Indicação de risco operacional.
- Comparação da localização atual com as cidades monitoradas.
- Sugestões ou alertas relevantes para aquela região.

5. Cidades monitoradas

Além da localização atual do usuário, o sistema deve acompanhar uma lista inicial de cidades brasileiras.

Cidades mínimas esperadas

- São Paulo
- Rio de Janeiro
- Belo Horizonte
- Curitiba
- Porto Alegre
- Florianópolis
- Brasília
- Salvador
- Recife
- Goiânia

O sistema deve permitir que essas cidades sejam exibidas no dashboard, comparadas entre si e usadas nas métricas gerais. Usuários com permissão adequada devem conseguir gerenciar as cidades monitoradas, incluindo adicionar, editar, ativar ou desativar cidades.

6. Login e controle de acesso

O sistema deve possuir uma pagina de login. Somente usuários autenticados devem conseguir acessar as áreas internas da aplicação.

O candidato deve criar tres tipos de usuários, com permissões diferentes.

Perfil	Pode acessar	Restrições principais
Administrador	Dashboard, agente, cidades monitoradas, usuários e área administrativa.	Acesso completo conforme o escopo.
Operador	Dashboard, dados climáticos, filtros, rankings, agente e cidades monitoradas.	Não altera permissões de usuários.
Visualizador	Dashboard, métricas, previsões e agente para dados disponíveis.	Não altera usuários, permissões ou configurações.

7. Tecnologias esperadas

A solução deve seguir a stack definida abaixo. O projeto pode ser desenvolvido como uma aplicação monolítica ou como um monorepo, desde que todo o código esteja organizado em um único repositório Git.

Camada	Tecnologia esperada
Frontend	React ou Next.js
Backend	Python
Banco de dados	PostgreSQL
Persistência	ORM do ecossistema Python, como SQLAlchemy, SQLModel, Django ORM ou equivalente
Agente	Python ou n8n, integrado com LLM via token/chave OpenAI fornecido
Repositório	Projeto monolítico ou monorepo em um único repositório Git

O backend deve utilizar PostgreSQL como banco de dados e um ORM para comunicação com o banco. O acesso ao banco não deve ser feito de forma desorganizada ou espalhada pela aplicação. Esperamos uma camada clara de persistência, respeitando a arquitetura proposta.

O agente climático deve utilizar obrigatoriamente uma LLM integrada ao sistema por meio do token de acesso fornecido para OpenAI. O token terá validade e poderá expirar, portanto o candidato deve considerar esse ponto na implementação e documentação.

8. Arquitetura esperada

Esperamos que o projeto seja estruturado com boas práticas de arquitetura.

- Princípios SOLID.
- Padrão de Clean Architecture.
- Separação clara entre regras de negócio, casos de uso, infraestrutura, integrações externas e camada de apresentação.
- Baixo acoplamento entre as partes do sistema.

- Código organizado, legível e fácil de evoluir.
- Validação adequada de entradas.
- Tratamento consistente de erros.
- Testabilidade das principais regras de negócio.

O candidato deve tomar as decisões técnicas necessárias para atender ao escopo, explicando no README as principais escolhas feitas. Não queremos apenas uma aplicação funcionando. Queremos entender como o candidato pensa, organiza responsabilidades e constrói uma base de código sustentável.

9. Usabilidade e experiência do usuário

A usabilidade é um ponto importante deste teste. A estética visual é bem-vinda e pode valorizar a entrega, mas o principal objetivo é avaliar se o sistema é fácil de usar, claro e funcional para o usuário final.

O candidato deve se preocupar com

- Facilidade para entender o dashboard.
- Clareza dos indicadores e métricas.
- Organização das informações na tela.
- Fluxo simples após o login.
- Feedback claro em carregamentos, erros e permissões.
- Facilidade para usar o agente climático.
- Facilidade para identificar riscos e alertas.
- Boa experiência para diferentes níveis de acesso.
- Interface funcional em diferentes tamanhos de tela.

Prioridade de UX

Não esperamos uma interface extremamente refinada visualmente. Esperamos uma experiência de uso bem pensada, simples, objetiva e agradável.

10. Dashboard principal

Após o login, o usuário deve ser direcionado para o dashboard principal. O dashboard deve apresentar uma visão consolidada do cenário climático das cidades monitoradas e da localização atual do usuário, quando disponível.

Indicadores esperados

- Total de cidades monitoradas.
- Quantidade de cidades ativas.
- Temperatura média geral.
- Cidade mais quente.
- Cidade mais fria.

- Localização atual do usuário, quando autorizada.
- Probabilidade média de chuva.
- Cidade com maior probabilidade de chuva.
- Cidade com maior volume previsto de chuva.
- Cidade com maior velocidade de vento.
- Quantidade de cidades em risco alto.
- Melhor cidade ou melhor período para operação externa.

Visualizacoes esperadas

- Comparação de temperatura entre cidades.
- Ranking de risco operacional por cidade.
- Comparação de probabilidade de chuva.
- Lista de alertas climáticos.
- Destáques da localização atual do usuário.
- Indicação visual para risco baixo, médio e alto.

11. Métricas climáticas

Métricas gerais

- Temperatura média das cidades monitoradas.
- Temperatura máxima prevista.
- Temperatura mínima prevista.
- Amplitude térmica.
- Probabilidade média de chuva.
- Maior probabilidade de chuva.
- Volume previsto de chuva.
- Velocidade média do vento.
- Maior velocidade de vento prevista.
- Classificação geral de risco.

Métricas por cidade

- Temperatura mínima.
- Temperatura máxima.
- Temperatura média.
- Probabilidade de chuva.
- Volume previsto de chuva.
- Velocidade do vento.
- Indicador de risco operacional.
- Previsão para os próximos dias.

Métricas para localização atual

- Temperatura da região atual.

- Probabilidade de chuva da região atual.
- Risco operacional da região atual.
- Comparação com as cidades monitoradas.
- Alertas relevantes para aquela localização.

12. Risco operacional

O sistema deve classificar as cidades e previsões em níveis de risco operacional: baixo risco, médio risco e alto risco.

A classificação deve considerar fatores como probabilidade de chuva, volume previsto de chuva, velocidade do vento, temperaturas extremas e combinação de fatores climáticos.

O candidato deve definir a regra de cálculo do risco e explicar essa decisão no README. O importante é que a regra seja clara, coerente e usada de forma consistente no dashboard, nos rankings e nas respostas do agente.

13. Página de cidades monitoradas

O sistema deve ter uma área para exibir as cidades monitoradas.

Informações esperadas

- Nome.
- Estado ou região.
- País.
- Status de monitoramento.
- Última atualização dos dados climáticos.
- Indicador de risco atual.

Usuários com permissão administrativa devem conseguir gerenciar essas cidades. Usuários sem permissão administrativa devem visualizar as informações conforme seu nível de acesso.

14. Página de previsão detalhada

O sistema deve permitir que o usuário analise uma cidade específica em mais detalhes.

- Previsão dos próximos dias.
- Temperatura mínima e máxima.
- Probabilidade de chuva.
- Volume previsto de chuva.
- Velocidade do vento.
- Classificação de risco por dia.
- Destaques importantes para tomada de decisão.

A localização atual do usuário também deve poder ser analisada nessa experiência, quando estiver disponível.

15. Agente climático dentro da interface

O sistema deve possuir um agente climático integrado a UI. Esse agente deve estar acessível dentro do sistema por meio de um botão, ícone, área lateral ou componente visual semelhante.

Quando o usuário clicar no agente, deve abrir um chat dentro da própria aplicação. O usuário deve conseguir digitar perguntas e visualizar as respostas diretamente nesse chat, sem sair do dashboard.

O agente deve responder com base nos dados climáticos carregados pelo sistema e deve usar obrigatoriamente uma LLM integrada por meio do token/chave da OpenAI fornecido.

Perguntas que o agente deve conseguir responder

- Qual cidade está mais quente?
- Qual cidade está mais fria?
- Qual cidade tem maior chance de chuva?
- Qual cidade tem maior volume previsto de chuva?
- Qual cidade tem maior velocidade de vento?
- Quais cidades estão com risco alto?
- Qual é o melhor lugar para uma operação externa?
- Qual é o melhor período para uma operação externa?
- Como está o clima na minha localização atual?
- Minha localização atual está em risco alto?
- Faça um resumo geral do clima das cidades monitoradas.
- Existe algum alerta importante para os próximos dias?
- Qual cidade tem maior variação de temperatura?
- Explique por que uma cidade foi classificada como risco alto.

Comportamento esperado do agente

- Responder de forma clara e objetiva.
- Usar os dados disponíveis no sistema.
- Informar quando não houver dados suficientes.
- Não inventar informações.
- Respeitar o nível de acesso do usuário.
- Ser útil para interpretar o dashboard.
- Funcionar integrado a experiência da aplicação, sem exigir que o usuário saia da interface.

Segurança do token

O token/chave da OpenAI deve ser tratado com segurança e não deve ser exposto publicamente no frontend ou no repositório. Caso o token expire ou exista qualquer complicação técnica, o candidato deve informar a situação o quanto antes.

16. Acesso administrativo

O sistema deve ter uma área administrativa simplificada, acessível apenas para usuários com permissão adequada.

Essa área deve aparecer no menu somente para usuários autorizados. Usuários sem permissão não devem visualizar o link, botão ou item de navegação relacionado a área administrativa.

Para este teste, a área administrativa não precisa ser complexa. O objetivo é validar se o candidato consegue controlar acesso por perfil e adaptar a interface conforme o nível de permissão do usuário.

17. Estádos e mensagens da interface

A aplicação deve tratar adequadamente os principais estados de uso. A experiência não deve quebrar quando algum dado externo não estiver disponível.

- Carregamento de dados.
- Erro de login.
- Usuário sem permissão para determinada ação.
- Falha ao obter localização atual.
- Permissão de localização negada pelo usuário.
- Falha ao consultar dados climáticos.
- Dados ainda não carregados.
- Cidade sem previsão disponível.
- Agente sem dados suficientes para responder.
- Token de agente ausente, inválido ou expirado, se aplicável.

18. Publicação da aplicação

O candidato receberá o IP do servidor e os dados de acesso necessários, incluindo usuário SSH e senha, para publicar a aplicação no ambiente disponibilizado para o teste.

Também será fornecida uma chave/token da OpenAI para integração do agente. Esse token terá validade e poderá expirar.

A aplicação deverá ser colocada no ar nesse servidor para que possa ser avaliada de forma online. A entrega não será considerada completa se o sistema funcionar apenas localmente.

O ambiente publicado deve permitir

- Acesso a aplicação pelo navegador.
- Login com os usuários de teste.
- Validação das permissões por perfil.
- Uso do dashboard.

- Uso da localização atual do usuário, quando autorizada no navegador.
- Uso do agente climático dentro da interface.
- Validação das principais funcionalidades descritas neste escopo.

Caso o candidato tenha qualquer complicação com o servidor, publicação, credenciais, token, integração do agente ou prazo, deverá nos avisar o quanto antes.

19. Repositório Git

O código do projeto deve estar em um repositório Git. A entrega não será considerada completa se o código não estiver disponível em um repositório Git acessível para avaliação.

O repositório será usado para avaliar

- Organização do projeto.
- Estrutura de pastas.
- Clareza do código.
- Separação de responsabilidades.
- Qualidade das decisões técnicas.
- Histórico de commits.
- Documentação de execução.
- Evolução da implementação.

20. Prazo de entrega

Data limite

O candidato terá até quarta-feira, 29/04/2026, às 23h59, para entregar a solução. A entrega deve incluir o repositório Git e a aplicação publicada no servidor informado.

Caso exista qualquer complicação que possa impactar a entrega, o candidato deve avisar o quanto antes, explicando o ocorrido de forma objetiva.

21. Entregáveis esperados

Entregáveis obrigatórios

A avaliação será feita sobre o código, a documentação, a aplicação publicada e a experiência de uso. Os itens abaixo devem estar disponíveis até o prazo final.

Código e documentação

- Link do repositório Git contendo todo o código do projeto.
- Código versionado em Git, com histórico de commits suficiente para avaliação.
- README com instruções claras para rodar o projeto.
- Explicação da arquitetura utilizada.
- Explicação de como aplicou SOLID e Clean Architecture.
- Explicação da regra de risco operacional.
- Explicação de como funciona o agente climático.
- Explicação de como a localização atual e solicitada é utilizada.
- Observações sobre decisões técnicas relevantes.

Acesso ao sistema

- Aplicação publicada e acessível online para avaliação.
- Link, IP, domínio ou endereço configurado para acesso a aplicação publicada.
- Usuários de teste com seus respectivos perfis.
- Informações necessárias para acessar e validar a aplicação online.

Ambiente e integração

- Aplicação funcionando no servidor informado.
- Backend em Python.
- Frontend em React ou Next.js.
- Agente integrado ao sistema em Python ou n8n.
- Uso seguro do token/chave da OpenAI fornecido.
- Uso de PostgreSQL no backend.
- Uso de ORM para comunicação com o banco de dados.
- Projeto organizado como monolito ou monorepo dentro de um único repositório Git.

22. Usuários de teste esperados

O candidato deve disponibilizar pelo menos três usuários para avaliação. As credenciais devem estar documentadas no README do projeto.

Usuário	Perfil
Administrador	admin
Operador	operator
Visualizador	viewer

23. Critérios de aceite

O sistema será considerado adequado se atender aos itens abaixo.

- Possuir tela de login funcional.
- Bloquear acesso as áreas internas sem autenticação.
- Implementar tres niveis de acesso.
- Solicitar permissão de localização no navegador após o login.
- Usar a localização atual quando autorizada.
- Continuar funcionando quando a localização for negada.
- Consumir dados reais da API climática pelo backend.
- Exibir dashboard com métricas climáticas relevantes.
- Comparar cidades monitoradas.
- Apresentar previsão detalhada por cidade.
- Classificar risco operacional.
- Ter agente em formato de chat dentro da interface.
- Ter agente implementado em Python ou n8n.
- Integrar obrigatoriamente o agente com LLM usando o token/chave da OpenAI fornecido.
- Responder perguntas básicas sobre os dados climáticos.
- Ter backend em Python.
- Ter frontend em React ou Next.js.
- Utilizar PostgreSQL como banco de dados no backend.
- Utilizar ORM para comunicação com o PostgreSQL.
- Manter todo o projeto em um único repositório Git, seja em formato monolítico ou monorepo.
- Apresentar organização coerente com SOLID e Clean Architecture.
- Ter boa usabilidade e experiência de uso.
- Ter README claro e suficiente para avaliacao.
- Estár publicado no servidor informado para o teste.
- Permitir validação online das principais funcionalidades do sistema.
- Ser entregue dentro do prazo informado.

24. Diferenciais

- Testes unitários das regras de negócio.
- Testes das regras de risco operacional.
- Testes de autenticação e permissão.
- Cache para evitar consultas externas desnecessárias.
- Boa modelagem e uso adequado do PostgreSQL.
- Uso adequado de ORM e camada de persistência bem organizada.
- Docker ou Docker Compose.
- Interface bem polida.
- Boa usabilidade e experiência de uso bem pensada.

- Gráficos claros e úteis.
- Histórico de consultas ou atualizações climáticas.
- Logs bem estruturados.
- Agente com boa interpretação de perguntas variadas.
- Deploy funcional no servidor informado.

25. O que não é obrigatório

- Recuperação de senha.
- Cadastro público de usuários.
- Integração com mapas.
- Notificações por e-mail, SMS ou WhatsApp.
- Machine learning avançado.
- Infraestrutura complexa em cloud.

26. Observações finais

Queremos avaliar a capacidade do candidato de transformar uma necessidade de negócio em uma solução funcional.

O foco está em

- Entendimento do problema.
- Backend em Python.
- Frontend em React ou Next.js.
- Integração com API externa.
- Arquitetura limpa.
- SOLID.
- Controle de acesso.
- Uso da localização atual do usuário.
- Cálculo de métricas.
- Classificação de risco.
- Clareza da interface.
- Usabilidade e experiência do usuário.
- Agente interno útil, preferencialmente em Python ou n8n.
- Integração obrigatória com LLM usando token/chave da OpenAI fornecido.
- Uso de PostgreSQL no backend.
- Uso de ORM para persistência.
- Organização como monolito ou monorepo em um único repositório Git.
- Organização e explicação das decisões técnicas.
- Código versionado em Git.
- Aplicação publicada para avaliação online.

- Entrega dentro do prazo.

O candidato deve propor a melhor forma de implementação e justificar suas escolhas. As decisões devem ser coerentes com o problema e bem explicadas.

O código deve estar em um repositório Git para que possamos avaliar organização, estrutura, evolução e qualidade da entrega.

Mensagem final

Não esperamos uma solução enorme. Esperamos uma solução objetiva, funcional, bem explicada e com decisões técnicas coerentes.